

Dutch National Flag Algorithm – Complete Notes

Contents

1	Problem	2
2	Brute Force	2
3	Observation	2
4	Core Intuition	2
5	Loop Invariant	2
6	Three Cases	2
7	Why Mid Is Not Incremented	3
8	Dry Run	3
9	Correctness	3
10	Complexity	3
11	Common Mistakes	3
12	Recognition Pattern	3
13	Implementation	4
14	Summary	4

1 Problem

Given an array containing only 0, 1 and 2, sort it in-place in one traversal using constant extra space.

Example:

$$[2, 0, 2, 1, 1, 0] \rightarrow [0, 0, 1, 1, 2, 2]$$

2 Brute Force

Count the number of 0s, 1s and 2s, then overwrite the array. Time: $O(n)$, Space: $O(1)$, but requires two passes.

3 Observation

The array has only three categories. We do not need comparison sorting. Instead, partition the array into regions.

4 Core Intuition

Maintain four regions:

$$[0, low - 1] = 0s, [low, mid - 1] = 1s, [mid, high] = Unknown, [high + 1, n - 1] = 2s.$$

Initially the entire array is Unknown. Every iteration shrinks the Unknown region until it disappears.

5 Loop Invariant

Before every iteration:

- Left of `low` contains only 0s.
- Between `low` and `mid-1` contains only 1s.
- Between `mid` and `high` is unprocessed.
- Right of `high` contains only 2s.

6 Three Cases

Case 1: `nums[mid]==0`

Swap with `low`, then increment both `low` and `mid`. The incoming value at `mid` is already known to be 1 (or the same element if `low==mid`).

Case 2: `nums[mid]==1`

Simply increment `mid` because 1 already belongs in the middle partition.

Case 3: `nums[mid]==2`

Swap with `high` and decrement `high` only. Do not increment `mid` because the swapped element from the right has never been examined.

7 Why Mid Is Not Incremented

Example:

[1, 2, 0]

Swap 2 and 0:

[1, 0, 2]

If mid were incremented, the 0 would never move into the left partition.

8 Dry Run

Step	low	mid	high	Array
0	0	0	5	[2,0,2,1,1,0]
1	0	0	4	[0,0,2,1,1,2]
2	1	1	4	[0,0,2,1,1,2]
3	2	2	4	[0,0,2,1,1,2]
4	2	2	3	[0,0,1,1,2,2]
5	2	3	3	[0,0,1,1,2,2]
6	2	4	3	Finished

9 Correctness

Initialization: invariant holds before first iteration. Maintenance: each of the three cases preserves the invariant. Termination: when $\text{mid} \geq \text{high}$, Unknown region is empty, so the whole array is partitioned.

10 Complexity

Time: $O(n)$ because every iteration shrinks the Unknown region. Space: $O(1)$.

11 Common Mistakes

- Incrementing mid after swapping with high.
- Using $\text{while}(\text{mid} < \text{high})$ instead of $\text{while}(\text{mid} \leq \text{high})$.
- Forgetting the loop invariant.

12 Recognition Pattern

Use this algorithm when:

- Exactly three categories exist.
- In-place partitioning is required.
- One-pass solution is expected.
- Three-way partition around a pivot.

13 Implementation

```
class Solution {
public:
    void sortZeroOneTwo(vector<int>& nums) {
        int low = 0;
        int mid = 0;
        int high = nums.size()-1;
        while(mid<=high){
            if(nums[mid]==0){
                swap(nums[mid], nums[low]);
                low++;
                mid++;
            }
            else if(nums[mid]==2){
                swap(nums[mid], nums[high]);
                high--;
            }
            else{
                mid++;
            }
        }
    }
};
```

14 Summary

Think in terms of shrinking the Unknown region, not sorting. Maintain the loop invariant throughout, and never increment mid after swapping with high.